

LAPORAN PRAKTIKUM 3
PEMOGRAMAN BERORIENTASI OBJEK
“KEAMANAN & INTEGRITAS DATA (ENKAPSULASI)”

DISUSUN OLEH :

DZHILLAN DZHALILA

2511531001

PRAKTIKUM KELAS C

DOSEN PENGAMPU:

NURFIAH, S.ST, M.KOM

ASISTEN PRAKTIKUM:

EZZA ADDINI



FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

A. TAHAPAN PELAKSANAAN PRAKTIKUM

1. Mengunci Entitas *Transaksi*

Langkah ini akan mengamankan data yang ada pada program dengan mengubah variabelnya menjadi **private** dan memberikan **getter**. Namun, objek transaksi itu bersifat *immutable* atau tidak boleh diubah setelah dibuat, sehingga tidak perlu membuat **setter**.

Source Code program Transaksi2

```
package Pekan3;

public class Transaksi2 {
    private String idTransaksi;
    private String jenis;
    private double nominal;

    public Transaksi2(String id, String jenis, double nominal)
    {
        this.idTransaksi = id;
        this.jenis = jenis;
        this.nominal = nominal;
    }

    public String getIdTransaksi () {
        return idTransaksi;
    }

    public String getJenis() {
        return jenis;
    }

    public double getNominal() {
        return nominal;
    }

    public void cetakDetail () {
        System.out.println("ID: " + idTransaksi + " |
Jenis : "+ jenis + " | Nominal : Rp" + nominal);
    }
}
```

Mengubah atribut menjadi kategori private akan menjadikan atribut hanya bisa diakses dari dalam Transaksi2 dan class lain tidak bisa mengakses. Agar bisa diakses lewat program lain, method publik (get) itu dibutuhkan untuk membaca data. Method cetakDetail() akan menampilkan data tanpa perlu class lain mengetahui isi atributnya.

Tidak adanya setter membuat data tidak akan dapat diubah sehingga integritas data terjaga.

2. Mengunci Entitas Rekening dan menambahkan PIN

Langkah ini akan mengamankan saldo dan menambahkan atribut PIN.

Source Program file Rekening3

```
package Pekan3;
import java.util.ArrayList;

public class Rekening3 {
    //1. Mengunci atribut dengan 'private'
    private String nomorRekening;
    private String namaPemilik;
    private double saldo;
    private String pin; // data sensitif

    private ArrayList <Transaksi> riwayatTransaksi;

    //2. Modifikasi Constructor untuk menerima PIN di awal

    public Rekening3 (String nomor, String nama, double
saldoAwal, String pinAwal) {

        //CHALLENGE NO 3 - validasi PIN
        if (!pinAwal.matches("\\d+")) {
            throw new IllegalArgumentException("PIN
harus berupa angka!");
        } else if (pinAwal.length() != 6) {
            throw new IllegalArgumentException("PIN
harus 6 digit!");
        } else if (isAngkaBerulang(pinAwal)) {
            throw new IllegalArgumentException("PIN
tidak boleh menggunakan angka yang berulang semua (contoh:
111111)!");
        } else if (isAngkaBerurutan(pinAwal)) {
            throw new IllegalArgumentException("PIN
tidak boleh menggunakan angka berurutan (contoh: 123456 atau
654321)!");
        }
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;
        this.pin = pinAwal;

        this.riwayatTransaksi = new ArrayList <>();
        System.out.println("Rekening atas nama " + namaPemilik + "
berhasil dibuat.");
    }

    //CHALLENGE NO 4 -- LARANGAN PIN ANGKA BERULANG DAN
ANGKA BERURUTAN
    //mengecek untuk digit yang berulang
```

```

        private boolean isAngkaBerulang(String
pin) {
            for (int i = 1; i <
pin.length(); i++) {
                if (pin.charAt(i) !=
pin.charAt(0)) {
                    return false;
                }
            }
            return true;
        }
        // mengecheck untuk digit berurutan naik atau
turun
        private boolean isAngkaBerurutan(String
pin) {
            boolean naik = true;
            boolean turun = true;

            for (int i = 1; i <
pin.length(); i++) {
                int sebelumnya =
pin.charAt(i - 1) - '0';
                int sekarang =
pin.charAt(i) - '0';

                if (sekarang != sebelumnya
+ 1) {
                    naik = false;
                }
                if (sekarang != sebelumnya
- 1) {
                    turun = false;
                }
            }
            return naik || turun;
        }

        //3. Getter untuk atribut yang diizinkan dibaca publik
        public String getNomorRekening () {
            return nomorRekening;
        }

        public String getNamaPemilik() {
            return namaPemilik;
        }

        //4. Method Otentikasi Internal (Validasi Enkapsulasi)
        public boolean otentikasi (String inputPin) {
            return this.pin.equals(inputPin);
        }

        public void setorTunai(double nominal) {
            if (nominal > 0) {
                saldo += nominal;
                String idTrx = "TRX-S-" +
System.currentTimeMillis();
            }
        }
    }
}

```

```

        Transaksi2 trxBaru = new Transaksi2
(idTrx, "Kredit", nominal);
        riwayatTransaksi.add(trxBaru);

        System.out.println("Setor tunai Rp" +
nominal + " berhasil. Saldo saat ini : Rp" + saldo);
    } else {
        System.out.println("Gagal : Nominal
setor harus lebih dari 0!");
    }
}

    public void tarikTunai (double nominal) {
        if (nominal > saldo ) {
            System.out.println("Transaksi Gagal : Saldo
Anda tidak cukup. Saldo Anda : Rp" + saldo);
        } else if (nominal < 10000) {
            System.out.println("Transaksi Gagal : Minimal
transaksi tarik tunai adalah sebesar Rp10000");
        } else {
            saldo -= nominal;
            String idTrx = "TRX-T-" +
System.currentTimeMillis();
            Transaksi2 trxBaru = new Transaksi2 (idTrx,
"Debit" , nominal);
            riwayatTransaksi.add(trxBaru);
            System.out.println("Tarik tunai Rp" + nominal
+ " berhasil. Saldo saat ini : Rp" + saldo);
        }
    }

    public void cekInformasi() {
        System.out.println("--- INFO REKENING ---");
        System.out.println("No. Rekening : " +
nomorRekening);
        System.out.println("Nama Pemilik : " +
namaPemilik);
        System.out.print("Saldo Akhir : Rp" + saldo);
        System.out.println();
        System.out.println("-----");
    }

    public void cekRiwayatTransaksi() {
        System.out.println("--- RIWAYAT TRANSAKSI ---");
        System.out.println("No. Rekening : " +
nomorRekening);
        if (riwayatTransaksi.isEmpty()) {
            System.out.println("Belum ada transaksi.");
        } else {
            for (Transaksi2 t : riwayatTransaksi) {
                t.cetakDetail();
            }
        }
        System.out.println("-----");
    }
}

```

Penjelasan : Pemberian private pada atribut yang tidak bisa diakses langsung dari luar. Hal ini melindungi data paling sensitif yaitu saldo dan PIN. Hanya dua data yang boleh dibaca publik yaitu yang menggunakan getter. Namun, untuk data kritis itu tidak memiliki setter seperti setSaldo dan setPin dimana hanya bisa berubah lewat setorTunai dan tarikTunai. Method ini akan lebih aman daripada setter biasa karena perubahan data selalu melewati validasi. Data rahasia berupa PIN memang tidak punya getter dan hanya bisa dicek lewat otentikasi().

B. LATIHAN/TUGAS

1. Meminta user untuk menginputkan PIN (String angka 6 digit)

```
//TUGAS NO 1
System.out.print("Masukkan PIN (6 Digit) : ");
String pin = input.nextLine();

try {
    Rekening3 rekeningBaru = new Rekening3(no, nama, saldo, pin);
    daftarRekening.add(rekeningBaru);
    akunAktif = rekeningBaru;
} catch (IllegalArgumentException e) {
    System.out.println("Gagal membuat rekening : " + e.getMessage());
}
break;
```

Penambahan kode ini membuat user harus memasukkan PIN berupa 6 digit string sebelum rekening bisa berhasil dibuat. Rekening bisa aktif ketika PIN sudah valid, jika PIN belum valid maka rekening belum bisa aktif dan menampilkan tampilan bahwa rekening gagal dibuat.

2. Memperbaharui Menu 3 (Tarik Tunai) dan Menu 6 (Cetak Mutasi)

```
case 3:
    if (akunAktif == null) {
        System.out.println("Error : Mohon maaf, Anda belum memiliki nomor rekening!");
    } else {
        // TUGAS NO 2
        System.out.print("Masukkan PIN : ");
        String pinTarik = input.nextLine();
        if (!akunAktif.otentikasi(pinTarik)) {
            System.out.println("Akses Ditolak : PIN yang anda masukkan salah!");
            break;
        }

        System.out.print("Masukkan nominal tarik : ");
        if (!input.hasNextDouble()) {
            System.out.println("Error : Input harus berupa angka");
            input.nextLine();
            break;
        }
        double tarik = input.nextDouble();
        input.nextLine();
        akunAktif.tarikTunai(tarik);
    }
    break;
```

Penambahan kode ini membuat user harus input PIN sebelum melakukan tarik tunai. Permintaan PIN adalah sebelum meminta berapa

nominal tarik. Jika PIN tidak sesuai dengan PIN yang diset pada pembuatan rekening, maka akses untuk melakukan transaksi ditolak.

```
case 6 :
    if (akunAktif == null) {
        System.out.print("Error : Anda belum membuka rekening");
    } else {
        //TUGAS NO 2
        System.out.println("Masukkan PIN : ");
        String pinRiwayat= input.nextLine();
        if (!akunAktif.otentikasi(pinRiwayat)) {
            System.out.println("Akses Ditolak : PIN yang anda masukkan salah!");
            break;
        }
        akunAktif.cekRiwayatTransaksi();
    }
    break;
```

Penambahan kode ini membuat user harus input PIN sebelum melihat riwayat transaksi. Jika PIN tidak sesuai dengan PIN yang diset pada pembuatan rekening, maka akses untuk menampilkan riwayat transaksi ditolak.

Output Program

```
=== SISTEM PERBANKAN MINI ===

Menu Utama:
Akun Aktif saat ini: -
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Riwayat Transaksi (Mutasi)
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 1001
Masukkan Nama Pemilik: Zaza
Masukkan Saldo Awal : 8900000
Masukkan PIN (6 Digit) : 098098
Rekening atas nama Zaza berhasil dibuat.
```

```

Menu Utama:
Akun Aktif saat ini: 1001 (Zaza)
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Riwayat Transaksi (Mutasi)
0. Keluar
Pilih menu: 3
Masukkan PIN : 098098
Masukkan nominal tarik : 100000
Tarik tunai Rp100000.0 berhasil. Saldo saat ini : Rp8800000.0

```

```

Menu Utama:
Akun Aktif saat ini: 1001 (Zaza)
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Riwayat Transaksi (Mutasi)
0. Keluar
Pilih menu: 3
Masukkan PIN : 098098
Masukkan nominal tarik : 40000
Tarik tunai Rp40000.0 berhasil. Saldo saat ini : Rp8760000.0

```

```

Menu Utama:
Akun Aktif saat ini: 1001 (Zaza)
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Riwayat Transaksi (Mutasi)
0. Keluar
Pilih menu: 6
Masukkan PIN :
098098
--- RIWAYAT TRANSAKSI ---
No. Rekening : 1001
ID: TRX-T-1790699805522 | Jenis : Debit | Nominal : Rp100000.0
ID: TRX-T-1790699813426 | Jenis : Debit | Nominal : Rp40000.0
-----

```

C. LINK REPOSITORY

https://github.com/zacodeif25/PrakPBO_C_2511531001/tree/Pekan_3/PrakPBO_C_2511531001/PRAKTIKUM